

Crew Ops Advisor Field Guide

The language model plans and explains. It never produces a fact and never does arithmetic.

587

tests passing

7

tests failing, all in resolve/

24

deterministic tools

100%

Tier 1 accuracy, offline, 4ms

16/16

Tier 1 via the agent, 0 wrong

0

verdict inversions anywhere

How to use this

This is the whole system written down: the dataset, the arithmetic, the architecture, the measured numbers, and the parts that do not work yet. It is ordered so that each part depends only on the ones before it. Sections 3 and 4 are the ones worth knowing cold, because every headline answer in the product falls out of them.

The sixty second version

A controller at an airline Crew Control desk asks a question in plain language. A LangGraph agent decides *which* deterministic computations to run and in what order, then phrases the result for someone under time pressure. Deterministic Python does every piece of computing. A verifier then checks every number, identifier, rule id, station code and date in the drafted reply against what the tools actually returned, and rejects anything unattested. When the system cannot answer reliably it declines, and says what was missing.

The boundary between those two halves is the submission. The problem statement puts 20% on exactly that question, and the interesting claim is not that the boundary exists but that it is *enforced structurally* rather than requested in a prompt.

Where the code lives

PATH	RESPONSIBILITY	MODEL?
data/	The provided dataset. Read only, never regenerated.	NO
contracts/	Shared types. The seam three workstreams built against.	NO

domain/	Typed records, loader, immutable worldState, copy-on-write overlay	NO
rules/	Clock arithmetic, the seven rules, RuleTrace	NEVER
ops/	Cover search, positioning, costing, ranking, simulation	NEVER
store/	Indexed SQLite projection of the world	NO
tools/	The 24 tools the agent calls	NO
agent/	LangGraph graph, prompts, guards, providers, memory	YES
verify/	The grounding verifier	NO
resolve/	Offline resolver, used when no API key is set	NO
eval/	The scorecard harness	NO
server/	FastAPI, SSE streaming	NO
web/	Next.js console. No answering logic, ever.	NO

Roughly 30,000 lines of Python and 14,000 of TypeScript. `tools/registry.py` is the largest single file at 3,582 lines; `rules/engine.py` is the densest at 1,103.

01 The thesis, and how it is enforced

The obvious approach to this problem is to put the dataset in a prompt and let the model answer. That fails at Tier 2 and Tier 3, because legality is exact arithmetic against a rulebook and a model that approximates a duty-hour calculation produces answers that are fluent, confident and wrong. Operationally that is worse than no answer at all.

The boundary, stated precisely

THE MODEL MAY

- Choose which tools to call, with which arguments, in what order
- Decide when it has enough to answer, and when it must abstain
- Decide how to phrase an answer for someone with a radio in one hand
- Decide which of the returned facts to surface first

THE MODEL MAY NOT

- State a number, identifier, date, station code, currency amount or rule id that no tool returned this turn
- Perform arithmetic, including "roughly", "about" and unit conversion
- Infer a rule verdict instead of calling `check_legality`
- Soften a breach into a warning, or an `insufficient_data` into a pass

Three independent enforcement mechanisms

None of the four prohibitions above depends on the prompt containing them. Each is enforced by code that can reject the turn.

<code>tests/test_boundary.py</code>	Walks the import graph of contracts, domain, rules, ops, store, tools and <code>verify</code> and fails if any model client is reachable at <i>any depth</i>. It does this twice: once with an AST walk over every source file, once by importing each package and checking nothing forbidden landed in <code>sys.modules</code>. It also asserts the mirror: that <code>agent/</code> <i>does</i> import a model client, on the grounds that a boundary with nothing on the other side of it is decorative.
<code>verify/</code>	A graph node, not a prompt line. Extracts every checkable atom from the drafted prose and matches it against the Fact objects the tools emitted this turn. Unattested atoms get one repair pass. A second failure abstains.
<code>agent/guards.py</code>	Five structural guards that ask a different question: was this answer <i>entitled to exist</i>. A legality claim with no rules-engine tool is refused regardless of whether its numbers check out.

There is a fourth, quieter one. `pyproject.toml` carries a ruff banned-api rule listing every model client, so importing one into the core is a lint error as well as a test failure. And `RuleId` is a closed `Literal` of exactly seven values, so an eighth rule is a type error rather than a judgement call.

THE ARGUMENT WORTH MAKING

The verifier checks **values**. The guards check **entitlement**. That split is not tidiness, it is a response to a failure the verifier structurally cannot catch.

Take the sentence `C-3310 is legal for P-2291`. Every atom in it is attestable: `C-3310` is a real crew id the tools returned, `P-2291` is a real pairing. The sentence can still be false, because *a verdict is a relation between values, not a value*. No amount of token matching catches a wrong relation. What catches it is refusing to accept a verdict that no rules engine produced.

The evidence model that makes it possible

Every tool returns a `ToolEnvelope`. Inside it, every figure is a Fact:

```
Fact(
  key="C-2087.duty_7d.projected",
  label="Projected 7 day duty",
  value=61.33,
  unit="hours",
  provenance=Provenance.COMPUTED,
  source="crewoos.rules.engine.LegalitvEngine".
```

```
derivation="51.83h prior + 9.50h from P-2291 = 61.33h "
    "against a 60.00h limit, over by 1.33h",
)
```

derivation is the point. A controller about to move a crew member and sign their name to it does not want to be told the answer, they want to check it and argue with it. It is also what makes the verifier possible: the derivation string is re-scanned, so the operands *inside* it are attested too.

The rule that holds the whole thing together: **if a number can appear in an answer, a tool must have emitted a Fact for it**. When the verifier fires, the fix is to add the missing Fact in the tool. It is never to relax the check.

02 The dataset, decoded

dCortex Air. Hub BLR. One week of schedule, 2026-09-14 to 2026-09-20 inclusive. All times UTC with a literal trailing Z. Currency INR. Snapshot ("now") is 2026-09-14T18:00:00Z.

docs/DATA-MODEL.md is the 1,600 line decoding, verified numerically against the shipped JSON. Read that rather than re-deriving from the files.

The eleven files

FILE	RECORDS	WHAT IT CARRIES, AND THE CATCH
flights.json	147	24 flight numbers over 7 dates, 21 legs per date. No leg crosses midnight UTC. block_hours equals arrival minus departure exactly.
crew.json	150	Rank, base, ratings, seniority, reachability, status. Names are not unique : seven are shared by two crew each.
rosters.json	39	Pairings over 42 duty days. report_utc is always first departure minus 60 min; release_utc is last arrival plus 30 min, for all 42.
duty_clocks.json	150	Summary 7d and 28d figures plus a 28 entry daily_history each. 4,200 history cells in total.
reserve_pool.json	16	On-call windows. All 16 are on call all 7 days , so the date never discriminates. The window is the only filter.
certifications.json	600	4 per crew. valid_from is unusable and must be ignored entirely.
rules.json	7	Three of the seven carry no params key at all, so a typed loader must make it optional.
costs.json	9	The rate card. No overtime rate exists, despite the problem statement describing one.
risk_signals.json	150	A provided disruption score, never computed by this system.
scenarios.json	6	Worked answer keys, S1 through S6.
questions.json	38	16 Tier 1, 14 Tier 2, 8 Tier 3, each with an expected answer.

The relational map

```
aircraft (6)    --< flight (147)
flight (147)   --1:1-- pairing-day leg slot (147)
pairing (39)   --< pairing-day (42)      1 day x36, 2 days x3
pairing-day    --< leg (147)            2 legs x7, 3 legs x7, 4 legs x28
pairing (39)   --< crew seat (206)      6 x25 A320, 4 x14 ATR72
crew (150)     --1:1-- duty_clock (150)
duty_clock     --< daily_history (4200) exactly 28 per crew
crew (150)     --< certification (600)  exactly 4 per crew
crew (150)     --1:1-- risk_signal (150)
crew (150)     --0..1-- reserve_pool entry (16)
```

Every referential check passes: zero orphans, zero dangling references. 102 of 150 crew appear on at least one pairing. Of the 48 who do not, 16 are reserves, 8 are on leave or in training, and 24 are spare active line crew.

NO REVERSE INDEX

There is **no crew to pairing index anywhere in the shipped data**. WorldState builds one at load time by scanning all 39 pairings, and calls it `week_duties`. Every window sum, every rest check and every overlap check downstream depends on it. This is the single most important derived structure in the system.

The two-day pairings, and why they exist

Only three pairings span two days: P-2291, P-2293 and P-2295. All three fly VT-DXC, which is the only tail with an alternating two-day rotation: BLR-BOM-BLR-DEL on odd dates, DEL-BLR-CCU-BLR on even ones. The aircraft overnights at DEL, which is why a cover has to take the whole remaining pairing rather than one day of it. Every hard case in the question set traces back to this.

P-2291, the flagship, in full

DAY	DATE	LEGS	ROUTE	REPORT	RELEASE	DUTY	BLOCK	FDP	LIMIT
1 of 2	2026-09-15	DX412/413/588	BLR-BOM-BLR-DEL	06:00Z	15:30Z	9.50h	6.25h		12.5h
2 of 2	2026-09-16	DX589/590/591	DEL-BLR-CCU-BLR	04:00Z	14:45Z	10.75h	7.75h		12.5h

Crew: C-1042 Captain, C-1694 First Officer, C-3005 Senior Cabin Crew, C-4395 / C-4273 / C-1873 Cabin Crew.

The anchor facts

Every figure quoted in the README, the deck and the demo comes from these. They are pinned by a dedicated golden test that forbids abstention on any of them.

C-1042 A. Nair, Captain, BLR, A320, seniority 22. Operates the two-day P-2291. The problem statement PDF says seniority 14. The PDF is stale. The data wins.

C-2087 Captain (the PDF calls them a First Officer and is wrong). Covering P-2291

C-3310	Reserve Captain, BLR, A320, window 06:00-18:00Z. Covers P-2291 cleanly at INR 18,500 and is the rank 1 option. Eligible only because the window test is inclusive: the window opens at 06:00 and the pairing reports at 06:00Z exactly.
C-3305	Reserve Captain. Legal for day 1 alone at 59.50h, breaches on day 2 at 68.25h. The best thirty seconds of the demo.
C-2210	The only DEL based Captain. Legal via deadhead at INR 41,200 with 3.0h of delay.
C-2091	ATR72 only. The canonical RULE-QUAL-05 exclusion from every A320 cover.
C-5417	The single flagged roster exception. <code>recurrent_training</code> expires 2026-09-17 and they are rostered on 2026-09-19.
C-5837	Fails on a rest conflict <i>two days after</i> the cover, invisible to any same-day check.

Where the problem statement and the data disagree

The PDF is stale in several places. The shipped data wins every time, and this table exists so a judge who checks a figure against the PDF sees that it was noticed rather than missed.

PDF SAYS	SHIPPED DATA SAYS
"FO C-2087" in the Tier 2 example	C-2087 is a Captain . Following the PDF builds a First Officer search for a Captain vacancy and returns an empty candidate list on the flagship scenario.
C-1042 <code>duty_hours_7d</code> : 48.5	20.93. Building from the PDF sample fails Q02, a mandatory Tier 1 question, on the first try.
C-1042 seniority: 14, no status field	22, and status is present and load bearing.
<code>reserve_pool.json</code> has "standby status"	No such field exists anywhere in the pack.
<code>costs.json</code> has "overtime" rates	There is no overtime rate.
"~140 flights", "~40 questions", "~8 stations"	Exactly 147, 38, 8. Quote the exact counts, never the approximations.

03 The clock arithmetic

This section is the foundation. Every seven-day and twenty-eight-day answer in the product is this formula, and two properties of it are counter-intuitive enough that a careful engineer will "fix" them and break every downstream answer.

The window function

```
def window_hours(crew_id, end_date, days, kind):    # "duty" or "flight"
    start = end_date - timedelta(days=days - 1)
    total = 0.0
    # 1. pre-snapshot accrual, dates <= 2026-09-14
    for entry in duty_clocks[crew_id].daily_history:
        if start <= entry.date <= end_date:
            total += entry.duty_hours if kind == "duty" else entry.flight_hours
    # 2. this week's roster, dates 2026-09-14 .. 09-20
    for wd in week_duties[crew_id]:
        if start <= wd.duty_date <= end_date:
            total += wd.duty_hours if kind == "duty" else wd.block_hours
    return round(total, 2)
```

Two things make this exactly right rather than approximately right.

The windows are calendar dates, not rolling hours

RULE-DUTY-02 is [end - 6, end] and RULE-FLT-03 is [end - 27, end], both inclusive UTC calendar dates, because rules.json says so in its own time_convention field. A rolling 168 hour clock gives different and wrong answers.

Round to two places at the end, not along the way

The reference implementation that produced the answer keys rounds every duration to two places at the point of measurement and again after summing a window. Rounding later, or not at all, walks the totals away from the shipped keys.

TRAP: THE 2026-09-14 DOUBLE COUNT

daily_history runs to 2026-09-14 inclusive and the roster week *starts* 2026-09-14, so that date contributes from **both** sources. Eleven crew are counted twice on it. It looks exactly like a bug and it is not.

Including the roster contribution reproduces duty_hours_7d for **150 of 150** crew. Dropping it reproduces only **118 of 150**, and 150 minus 118 is 32, which is exactly the number of crew rostered on 09-14. The dataset's own validate.py asserts the double-counted figure.

"Fixing" it moves 32 crew's totals, takes C-2087 off 61.33h, and turns the flagship breach into a pass. Three separate tests in tests/core/test_clocks.py pin all three numbers so nobody can quietly correct it.

Simulating a cover, and the cumulative add

When testing whether candidate X can cover pairing P, for **each** day d of the cover:

```
base = window_hours(X, d, 7, "duty")
      - sum(X's own duties on the pairing being replaced, inside the window)

added = sum(duty hours of every cover day whose date <= d)    # CUMULATIVE
```

```
total = round(base + added, 2)
breach <=> total > 60
```

The added term being cumulative is the whole game. On day 2 of a two-day cover, day 1's cover duty is *already inside* the seven-day window and must be counted again.

Worked: why C-3305 fails

C-3305 covering P-2291

```
2026-09-15  base 50.00 + cover 9.50          = 59.50  PASS  (0.50h spare)
2026-09-16  base 48.00 + cover 9.50 + 10.75   = 68.25  BREACH by 8h15m
```

Counting only day 2's own duty gives 48.00 + 10.75 = 58.75 and wrongly passes.

An implementation that checks the first day of a multi-day pairing and stops returns "yes, C-3305 can cover it" with correct arithmetic for the day it checked. Nothing about that answer looks like a partial check.

Worked: why C-2087 fails twice

C-2087 covering P-2291 (holds no rostered duty this week at all)

```
2026-09-15  51.83 + 9.50          = 61.33  over by 1.33h -> "1h20m"
2026-09-16  40.83 + 9.50 + 10.75  = 61.08  over by 1.08h -> "1h05m"
```

Both strings appear verbatim in the shipped answer for Q18 and for scenario S2. A collapsed single verdict loses the second line.

The defence is structural, not careful

LegalityReport.overall is defined as the **worst day and never an average**, per_day is a list rather than a scalar, and the contract package carries the invariant in writing. A single-day verdict for a two-day pairing is not representable in the type.

Duration rendering

The shipped keys render decimal hours in h/m form, and the conversion truncates whole hours then rounds the remainder to the nearest minute.

```
format_duration(1.33)  -> "1h20m"    # 0.33 x 60 = 19.8, rounds to 20
format_duration(8.25)  -> "8h15m"
format_duration(1.08)  -> "1h05m"
format_duration(12.0)  -> "12h00m"
```

This is why the verifier's normaliser canonicalises durations to whole minutes: 61.33, 61.33h, 61h20m and "61 hours 20 minutes" all reduce to 3680 minutes and are therefore one fact.

04 The seven rules

Seven is the complete regulatory scope. There is no eighth. RuleId is a closed Literal, so adding one is a type error.

Every rule produces a RuleTrace carrying a complete arithmetic string: both operands, the operator, the result, the limit and the signed margin. Not a summary, a statement.

The comparison directions

Each of these flips a headline answer if written the other way round. Float slack is EPSILON = 1e-6, applied as observed > limit + EPSILON and rest < limit - EPSILON, so a value sitting exactly on a limit is legal in both directions.

RULE	BREACH TEST	THE CASE THAT PROVES IT
RULE-FDP-01	fdp > limit	DX454-2026-09-17 is rated "delay (crew legal)" at exactly 12.0h against a 12.0h limit
RULE-DUTY-02	total > 60	A 60.00h projection is legal
RULE-FLT-03	total > 100	Never binds. The dataset peak is 79.28h.
RULE-REST-04	rest < 12	Exactly 12.0h of rest is legal
RULE-QUAL-05	type not in ratings	C-2091 is ATR72 only
RULE-CERT-06	valid_to < duty_date	C-5417's cert expires 09-17, so their 09-17 duty is legal and only 09-19 is not
RULE-BASE-07	different base, no positioning	Only DEL to BLR positioning exists in this schedule

RULE-FDP-01, maximum flight duty period

```
fdp_limit(n) = 13.0 - 0.5 * max(0, n - 2)
fdp_actual   = release_utc - report_utc, in hours
breach       <=> fdp_actual > fdp_limit           (strict)

sectors:  2 -> 13.0h    3 -> 12.5h    4 -> 12.0h    5 -> 11.5h
```

Only sector counts 2, 3 and 4 occur in this dataset. Real output from the engine:

```
PASS Report 06:00Z to release 15:30Z is 9.50h against a 12.50h limit
      for 3 sectors (13.0h base minus 0.5h per sector beyond 2), 3.00h spare
```

TRAP: DROPPING A LEG RAISES THE LIMIT

Fewer sectors means a *higher* FDP limit. In scenario S4 a 90 minute delay runs the four-sector duty to 12.75h against a 12.0h limit. Dropping the last leg leaves three sectors at 9.5h against a **12.5h** limit, which the rostered crew can legally fly. Reusing the original limit would call the partial duty illegal too and push a controller straight to cancellation, which costs INR 250,000 instead of 75,000.

RULE-DUTY-02, sixty duty hours per rolling seven calendar days

Covered in full in section 3. The failure message format the keys use:

```
excess = total - 60
f"RULE-DUTY-02: would exceed 60h/7d by {format_duration(excess)} "
f"on {duty_date} (total {total}h)"

-> RULE-DUTY-02: would exceed 60h/7d by 1h20m on 2026-09-15 (total 61.33h)
```

RULE-FLT-03, one hundred block hours per rolling twenty-eight days

Identical shape to DUTY-02 with `flight_hours` and `block_hours`.

SAY THIS BEFORE A JUDGE FINDS IT

This rule **never binds anywhere in the dataset**. The maximum 28-day block total across all 150 crew and all seven dates is 79.28h against a 100h limit. It appears in every `rules_checked` array in every answer key, which makes coverage look complete when it is not. It is implemented and evaluated like the others, and no shipped question can drive it to a breach, so its failure message format is unverified against the keys.

RULE-REST-04, minimum twelve hours rest

```
for consecutive duty pairs (a, b) sorted by report time:
    rest = (b.report_utc - a.release_utc) in hours
    breach <=> rest < 12                                (strict)
    overlap <=> b.report_utc < a.release_utc             (shows as negative rest)
```

Two things here are easy to get wrong and both change the ranked list.

It must be checked forward as well as backward. A rest check that only looks back at `last_rest_ended` wrongly passes C-5837, C-2143 and C-3187 and produces the wrong ranked list on the flagship scenario. The conflict for C-5837 is *two days after* the cover:

```
RULE-REST-04: only 10.75h rest before P-2204 on 2026-09-17 (downstream conflict)
```

The direction is labelled. The shipped strings distinguish (`rest conflict`), meaning the candidate's existing duty runs too close *before* the cover, from (`downstream conflict`), meaning the cover runs too close before one of their later duties. A controller needs to know whether the collision is behind or ahead.

A negative rest value is not a bug, it is the encoding of an overlap, and the reference implementation reports *both* a rest issue and a separate double-booked: issue when duties overlap.

RULE-QUAL-05, aircraft type rating

```
actype = flights[first leg of first day].aircraft_type
```

```
legal <=> actype in crew.ratings
```

TRAP: IT SHORT-CIRCUITS

A rating failure **suppresses every other reason** for that candidate. The shipped exclusion strings show "RULE-QUAL-05: no ATR72 rating" alone, with no accompanying rest or duty reason, even where those also apply. Emitting every reason produces text that does not match the keys even when the verdict does. RULE-BASE-07 short-circuits ahead of it on the same principle.

RULE-CERT-06, certification validity

```
legal <=> for every one of the crew's 4 certifications:
    cert.valid_to >= duty_date          (inclusive)
```

A certificate expiring *on* the duty date is valid that day. This is what makes C-5417 legal on 2026-09-16 and illegal on 2026-09-19 with a recurrent_training expiry of 2026-09-17.

TRAP: VALID_FROM IS GARBAGE

It is generated as valid_to - 730 days and never corrected after the engineered expiries were written in. One record (C-2087's licence) has valid_from 2028-11-06 and valid_to 2026-09-18. Several show a future start date for a currently flying crew member. **Check valid_to only.**

Two mechanisms enforce this. A test asserts the literal string .valid_from does not appear in rules/engine.py, and the SQLite projection does not carry the column at all: a column nobody can read is a column nobody can misread.

RULE-BASE-07, base and deadhead positioning

```
base_needed = flights[first leg of first day].dep_station
deadhead    = (crew.base != base_needed)
```

If the crew member is based elsewhere, a positioning flight must exist on the cover date, and the cost applies. The full positioning model is in section 5.

Evaluation order, and why it matters

assess_cover mirrors the reference implementation exactly, because the order determines *which reason is reported* when several apply:

1. RULE-BASE-07. Short-circuits when no positioning exists.
2. RULE-QUAL-05. Short-circuits everything below.
3. Per cover day: RULE-CERT-06, then RULE-FDP-01 on the delayed duty if positioning applies.
4. Own duties merged with the cover, sorted by report time: RULE-REST-04 pairwise, then double bookings as a separate pass.

double bookings as a separate pass.

5. Per cover day: RULE-DUTY-02 with the cumulative add, then RULE-FLT-03.

Three verdicts that are not PASS

PASS	The rule was evaluated and cleared. PASS
BREACH	The rule was evaluated and failed. BREACH
NOT_APPLICABLE	The rule was evaluated and had nothing to measure, for example REST-04 on a date with no adjacent duty. The reason is always stated. NOT APPLICABLE
INSUFFICIENT_DATA	The rule was <i>not reached</i> , or its inputs are missing. A short-circuited candidate's remaining rules land here. INSUFFICIENT DATA

Neither of the last two is PASS, and the contract package says so in writing. A rule that simply did not appear in a day's traces would read as a passing rule, so `_complete()` guarantees all seven are present on every day, in rule id order, filling gaps with an explicit NOT_APPLICABLE and its reason.

05 Cost, ranking, positioning

The rate card

KEY	INR	APPLIES WHEN
reserve_callout_pilot	18,500	Candidate is in the reserve pool, rank Captain or First Officer
reserve_callout_cabin	9,500	Candidate is in the reserve pool, cabin rank
dayoff_callout_pilot	24,000	Not a reserve, pilot rank
dayoff_callout_cabin	12,500	Not a reserve, cabin rank
deadhead_positioning	6,500	Per positioning, when the base differs from the departure station
delay_cost_per_duty_hour	5,400	Per hour the duty's first departure is delayed
cancellation_per_flight	250,000	Per leg, not per pairing
hotel_overnight	4,200	Never charged in any shipped answer key

THREE PRICING TRAPS

Callout is charged once per assignment, not per duty day. Covering the two-day P-2291 with reserve C-3310 costs 18,500, not 37,000.

hotel_overnight is never applied, including for the DEL overnight on the two-day pairings. It exists in the rate card and charging it would move every multi-day cover

price away from the keys.

Cancellation is per leg. Six legs of P-2291 is 1,500,000, not 250,000.

Positioning, derived rather than hardcoded

The rule implemented is "the earliest arrival into the required station, from the crew member's base, on the cover date", then:

```
new first departure = positioning arrival + 75 min    (15 min transit + 60 min report lead)
new report          = positioning arrival + 15 min
delay_hours         = max(0, new first departure - original first departure)
```

The 75 minute constant is load bearing: getting it wrong shifts the delay hours and therefore the cost.

Worked: C-2210 covering P-2291

```
C-2210  Captain, base DEL, A320, reserve, window 03:00-15:00Z
P-2291  day 1 2026-09-15, first leg DX412 dep BLR 07:00Z, report 06:00Z

base_needed BLR, crew base DEL          -> deadhead
2026-09-15 is odd, DX589 does not operate -> position on DX402, arr BLR 08:45Z
new first departure 08:45 + 75 min        = 10:00Z
delay_hours         10:00 - 07:00        = 3.0h
new required report 06:00 + 3.0h         = 09:00Z
window test         03:00 <= 09:00 <= 15:00 -> eligible

cost = reserve_callout_pilot      18,500
      + deadhead_positioning      6,500
      + round(3.0 x 5,400)        16,200
      -----
                                INR 41,200 matches the shipped key exactly
```

ONE DELIBERATE DIVERGENCE FROM THE REFERENCE

The reference implementation hardcodes `base == "DEL"` and `required == "BLR"` and refuses every other cross-base request outright. This implementation derives the answer from the schedule, so a BLR based crew member asked to cover a DEL origin duty gets a real positioning plan rather than a blanket refusal.

No shipped answer key exercises that branch, and the date parity falls out for free: on even dates it selects DX589 (arrives BLR 07:45Z), on odd dates DX402 (08:45Z), which is exactly the reference's behaviour. The generalisation is safer against held-out material than the special case would be.

Reserve on-call windows

- Tested against the **required report time**, never the scenario's narrative `reported_utc`. S1's callout is 01:30Z but the required report is 03:00Z, and the shipped exclusion string quotes 03:00Z.

- **Inclusive at both ends.** C-3310's window opens at 06:00 and P-2291 reports at 06:00Z, which is why C-3310 is the expected choice rather than an exclusion.
- After a deadhead, tested against the **delayed** report. C-2210 is tested at 09:00Z, not 06:00Z, and that changes who is eligible.
- **Only day 1 is window-tested.** Once activated a reserve operates as line crew, so day 2 of a multi-day cover has no window test.
- No window wraps midnight in this data, so a same-date interval test is correct and wraparound logic would be inventing a case that does not exist.

Ranking

```
sort legal options by (cost_inr, crew_id) ascending
append the cancellation option LAST, regardless of its cost
rank = index + 1
```

Cancellation is a last resort, not a cheap one, and the reference appends it after sorting. Every `CoverOption` also carries stated tradeoffs, and there is a test asserting the list is never empty: an option with no stated downside is under-analysed, not perfect.

Joint allocation across simultaneous gaps

Two captains calling in sick on the same morning is not two independent problems. There is one reserve captain whose window covers both early reports, and assigning them to one pairing removes them from the other.

```
enumerate the option lists for every gap
forbid the same crew_id appearing twice
minimise the total cost
cancellation is always available, so a plan always exists
```

Scenario S6's optimum is **INR 42,500**: C-3305 at 18,500 plus any one of ten day-off captains at 24,000. The dataset's own note says equal-cost mirror assignments are equally correct, so `JointPlan.alternatives` carries the other optimal plans rather than pretending one arbitrary pick is the answer.

WHY THIS TOOL EXISTS AT ALL

Calling `find_cover_options` twice returns **C-3305 as rank 1 for both gaps**, because C-3305 genuinely is the cheapest legal cover for each considered on its own.

Composing the two answers assigns one person to two aircraft departing thirty minutes apart, and reports a total of INR 37,000 against a true 42,500.

This is the only failure mode in the whole question set that produces a *confident wrong operational instruction* rather than a refusal. Every other gap causes an abstention. A controller acting on it sends two callouts to the same person and finds out when one aircraft has no captain at the gate.

06 Candidate enumeration

This is the single most reusable algorithm in the dataset, reconstructed from the answer keys and confirmed against the generator. The order of the filters is load bearing, because it decides which reason a candidate is excluded for when several apply.

```
for each crew c in crew.json:
    skip if c.crew_id == the crew member who is out
    skip if c.crew_id in the caller's forbid list
    skip if c.rank != the required role          # exact match, SCC != CC
    skip if c.status != "active"                # silently, never listed

    is_reserve = c.crew_id in reserve_pool
    if c.base != required departure station:
        plan a positioning flight
        if none exists: EXCLUDE "RULE-BASE-07: no same-day positioning flight from base"

    if is_reserve:
        required_report = day1.report_utc + delay_hours
        if not (window_start <= required_report <= window_end):
            EXCLUDE "reserve on-call window HH:MM-HH:MMZ does not cover required report HH:MMZ"

    run the full seven rule assessment # QUAL-05 short-circuits
    if any issue: EXCLUDE "; ".join(issues)
    else: price it and add to options

sort options by (cost_inr, crew_id)
append cancellation LAST
rank = index + 1
```

TRAP: NON-ACTIVE CREW ARE DROPPED SILENTLY

The eight leave and training crew are filtered *before any rule runs* and never appear in the exclusion list. They are not rule failures. Reporting them would put eight names in every exclusion list that the shipped keys do not have.

TRAP: RANK EQUALS ROLE, EXACTLY

Senior Cabin Crew is not substitutable for Cabin Crew, in either direction. Scenario S5 is a Cabin Crew cover and the callout rate differs by role (9,500 against 18,500), so getting this wrong returns the wrong candidate population at the wrong price.

Real output: the flagship cover search

```
ops.find_cover_for_pairing("P-2291", role="Captain", sick_crew_id="C-1042")
```

```
candidates evaluated: 24   excluded: 19   legal options: 5   (+ cancellation)
```

rank 1	C-3310	INR	18,500	delay 0.0h	reserve callout
rank 2	C-1526	INR	24,000	delay 0.0h	day-off callout
rank 3	C-3983	INR	24,000	delay 0.0h	day-off callout
rank 4	C-5566	INR	24,000	delay 0.0h	day-off callout
rank 5	C-2210	INR	41,200	delay 3.0h	reserve callout + deadhead from DEL

And the rejects, which are half the answer:

```
C-1600  RULE-QUAL-05: no A320 rating
C-1671  RULE-QUAL-05: no A320 rating
C-2087  RULE-DUTY-02: would exceed 60h/7d by 1h20m on 2026-09-15 (total 61.33h);
        RULE-DUTY-02: would exceed 60h/7d by 1h05m on 2026-09-16 (total 61.08h)
C-1443  RULE-REST-04: only -6.25h rest before COVER on 2026-09-15 (rest conflict);
        double-booked: P-2216 overlaps COVER on 2026-09-15
C-1017  RULE-REST-04: only 11.0h rest before P-2217 on 2026-09-16 (downstream conflict)
```

Every one of the 19 exclusions and every reason string reproduces the shipped key exactly, verified by tests/core/test_cover.py.

THE ONE THING THAT CANNOT BE MATCHED POSITIONALLY

The shipped excluded_candidates lists are ordered by the generator's internal crew creation order, which puts the ten engineered crew first. crew.json is written **sorted by crew id**, so that ordering is not recoverable from the shipped data.

Every excluded crew member and every reason string reproduces, for all five cover searches in the scenarios (18, 19, 21, 12 and 12 exclusions). Only the sequence differs. The test asserts the *mapping* rather than the list, and documents why. That is worth saying out loud rather than letting a judge notice a list in a different order.

The eighth constraint that is not a rule

The shipped keys exclude candidates for a reason the rulebook does not cover: double-booked: P-2203 overlaps COVER on 2026-09-16. A person cannot be in two places at once. That is an assignment feasibility check, not a regulation.

Modelling it as an eighth RULE- id would misrepresent the rulebook to someone who has to defend the decision, so it is carried as a FeasibilityIssue instead: real, blocking, and honestly labelled as operational rather than regulatory. Its code is lowercase and never starts with RULE-, it never appears in a rules_checked list, and the contract enforces that with a closed Literal of feasibility codes.

This is also why LegalityReport has two properties rather than one. is_legal is regulatory legality alone. is_assignable is legal *and* operationally possible, and that, not the first one, is the question a controller is actually asking.

07 The package map

contracts/ — the seam

Pure Pydantic types. Zero behaviour, zero model imports, and a local CLAUDE.md saying "do not edit this package without saying so". Three workstreams built against it in parallel: domain/rules/ops/store/tools produce these types, agent and verify consume them, server serialises them and web/src/lib/contracts.ts mirrors them.

Six invariants live here, and each has a test:

- 1. No model client import in this package, ever.
- 2. No behaviour. Validators and derived properties are fine; an `if` that encodes a rule belongs in `rules/`.
- 3. Every numeric payload field has a matching `Fact`.
- 4. `Verdict.INSUFFICIENT_DATA` is not `Verdict.PASS`. Silence about a rule is not compliance with it.
- 5. `LegalityReport.overall` is the worst day, never an average.
- 6. `TOOL_NAMES` stays in sync with the `ToolSurface` protocol methods.

domain/ — typed records and the immutable world

Every record is frozen with `extra="forbid"`. Timestamps parse to **naive** datetimes understood as UTC, because `Pydantic` would otherwise read the trailing `Z` as an offset and hand back an aware value that stops comparing against everything else in the system.

`WorldState` holds every shipped record plus the indices the data does not carry, built once at startup and never mutated. `WorldOverlay` is a cheap copy-on-write layer: `with_absence`, `without_pairing`, `with_duties`, each returning a *new* overlay. Every simulation runs on one, so two simulations can never contaminate each other and the base is always the shipped truth.

rules/ — the legality kernel

1,103 lines. Reads its thresholds off the loaded `RuleBook` with the constants in `limits.py` as a fallback, so a change to `rules.json` is honoured rather than silently ignored.

`messages.py` holds the terse exclusion strings in the exact shape the shipped keys use, separate from `RuleTrace.arithmetic` which is the full working. Both are produced at the same point in the engine so they cannot drift apart. The float rendering is Python's default `str(float)`, which is what produced `10.75h` and `12.0h` in the keys.

ops/ — search, costing, ranking, simulation

<code>candidates.py</code>	The enumeration in section 6, plus the projection into contract-shaped <code>CoverOption</code> and <code>Recommendation</code> values.
<code>positioning.py</code>	Deadhead planning, derived from the schedule.
<code>costing.py</code>	The rate card applied. Every <code>CostLine</code> carries a basis showing the multiplication, not just a total.
<code>disruption.py</code>	851 lines. Absence, reassignment, station closure and the two delay models.
<code>joint.py</code>	Exhaustive allocation with a distinctness constraint, capped at 200,000 combinations before falling back to a greedy pass that says it is greedy.

The two delay models

Both appear in the shipped keys and they answer different questions. Using the wrong one produces a plausible number and the wrong verdict.

MODEL	WHERE	ARITHMETIC
Release slides, report does not	S3, Q35: a closure part way through a duty the crew already reported for	<code>fdp_after = original duty length + delay</code>
The whole duty slides	S4, Q20: a technical delay before the first departure	<code>report and release both move, so the length grows too</code>

Station closure

```
affected <=> (dep_station == S and w_start <= dep_utc < w_end)
              or (arr_station == S and w_start <= arr_utc < w_end)

anchor          = dep_utc if departing the closed station in window else arr_utc
min_delay_hours = (w_end + 30 min) - anchor
crew_fdp_after_delay = original duty length + min_delay_hours
action          = "delay (crew legal)" if fdp_after <= limit else infeasible
```

The window is **half-open**, [start, end): a movement exactly at the reopen time is not affected. That reproduces S3's 13-flight set exactly, including DX412-2026-09-17 departing BLR at 07:00 being unaffected even though it shares a tail with an affected leg.

A BYTE-LEVEL DETAIL

The shipped infeasible action string separates its two clauses with a **U+2014 em dash**. This repository bans em dashes in authored source, so `disruption.py` builds the string from an escape ("`-`") with a comment explaining why. The output matches the key byte for byte without an em dash appearing in code anyone wrote.

store/ — the SQLite projection

WorldState stays the source of truth in memory. This is a *projection*: built from the world at startup, never written back to, with real indices on the filters that discriminate. It is the honest answer to "does this approach scale": the tool surface is a query interface, so swapping SQLite for a real operational store touches the store, not the engine.

It refuses to be built inside `data/` and there is a test for both the direct and nested cases.

tools/ — the boundary

3,582 lines implementing ToolSurface. Four rules every method follows:

- 1. Every numeric value in the payload has a matching Fact.
- 2. Computed facts carry the arithmetic, written out.

- 3. `ok=False` with a specific error when a lookup *fails*. An empty result is a *finding* and comes back `ok=True`.
- 4. Payloads stay inside a prompt budget, and set `truncated=True` when capped rather than silently losing rows.

RULE 3 IS THE ONE THAT MATTERS

"No crew match that filter" and "that crew id is not in the dataset" are different answers and must never look the same. `costs.json` has a `hotel_overnight` rate and no hotel bookings, so "what will the hotel cost" is answerable and "is the hotel booked" is not. A system that answers the second because it recognised the word "hotel" is the dangerous version.

08 The tool surface

Twenty-four tools across three tiers plus cross-cutting. The agent binds exactly these names and never reaches past them.

Tier 1, retrieval

<code>find_crew</code>	Crew matching base, rank, rating, reserve date, availability, name substring, status.
<code>get_crew_detail</code>	One crew member fully resolved: roster, duty clocks with headroom, certifications with validity, reserve window, and the provided risk signal. The tool the agent reaches for when a question names a person.
<code>find_flights</code>	Flights by origin, destination, date, time window, pairing, aircraft type or tail registration .
<code>find_pairings</code>	The route from an aircraft tail or a single leg <i>back</i> to the pairing that contains it. Without this a question naming VT-DXB is unanswerable.
<code>get_duty_clocks</code>	Duty and flight hour state with remaining headroom under each limit. Returns both the consumed figure and the headroom, because "how many hours has X got left" is a headroom question.
<code>list_reserves</code>	Reserves for a date with on-call windows. Window containment is inclusive both ends and tested against the required report time.
<code>find_expiring_certifications</code>	Licences, medicals and recurrent training lapsing inside a window. <code>valid_to</code> only.
<code>get_pairing</code>	A pairing with every duty day, leg, report and release time, FDP limit and the crew

assigned.

<code>get_roster</code>	One crew member's assignments across a date range, plus their days off.
<code>find_crew_at_risk</code>	Crew ranked by the provided disruption risk signal. Never modelled here.
<code>aggregate</code>	Count, sum, max, min, mean, distinct over a collection, with optional grouping and filters. This exists precisely so the model never does it: "how many", "which is longest" and "which stations" are aggregations, and aggregation is arithmetic.
<code>get_cost_rates</code>	The rate card as shipped, so "what does a callout cost" never comes from a model recalling a number.

Tier 2, consequence

<code>check_legality</code>	All seven rules for a crew member taking an assignment. Takes <code>crew_ids</code> to check several people against the same assignment in one call . For a multi-day pairing it returns one <code>DayLegality</code> per day and the overall verdict is the worst day.
<code>simulate_absence</code>	Which flights are now uncrewed, which pairings broke, how many passengers are exposed, and which other crew move closer to a limit.
<code>simulate_reassignment</code>	Moving a crew member onto an assignment: checks the mover, anyone displaced, and the downstream legs.
<code>simulate_station_closure</code>	Half-open window, per-flight minimum delay, post-delay crew FDP against the recomputed limit, and an action per flight.
<code>simulate_delay</code>	One flight running late, with a <code>mode</code> selecting which of the two delay models applies. The two give different answers, so the caller's choice matters.
<code>scan_duty_headroom</code>	Fleet-wide sweep for crew approaching a limit, in one call rather than 150.
<code>earliest_report</code>	RULE-REST-04 read forwards: when may this crew next report. Adding twelve hours to a timestamp is exactly the arithmetic the boundary keeps out of a model, so without this tool the question must be declined.

Tier 3, recommendation

<code>find_cover_options</code>	Enumerate, rule check, price and rank every way to cover a gap. Returns the rejected candidates each with the trace that excluded them. Requires
---------------------------------	---

for_crew_id or role: a pairing carries six crew across four ranks, so "cover P-2291" is ambiguous until you say whose seat is empty.

<code>plan_joint_cover</code>	Two or more simultaneous gaps as one allocation, with an explicit contention list naming any candidate that was rank 1 for more than one gap and how the conflict was resolved.
<code>draft_notification</code>	A deterministic template filled from computed facts. The agent may adjust tone. It may not introduce a time, flight number or report location the template did not supply.

Cross cutting

<code>get_watchlist</code>	The proactive brief for a date, ranked by severity.
<code>get_world_summary</code>	Dataset shape, snapshot, stations, fleet, and a coverage note that lets the system say honestly what it does not cover.
<code>explain_rule</code>	The machine-readable definition of one rule as shipped, with a worked example drawn from a case the answer keys settle, so the model never has to invent one.

Two classifications the graph enforces

RETRIEVAL_ONLY names the 14 tools that cannot on their own support a Tier 2 or Tier 3 answer. Retrieval establishes what *is*; it does not establish what *follows*.

REQUIRED_FOR maps a claim type to the tools that can stand behind it. `guards.py` derives its own sets from this dictionary rather than restating them, and a test fails if anyone restates them, because these were two hand-maintained lists and they had already drifted: `guards.py` carried `simulate_reassignment` and not `plan_joint_cover`, and `REQUIRED_FOR` carried the reverse. Nothing failed. The symptom, much later, was a guard refusing an answer the rules engine had genuinely produced.

09 The agent graph

```
START -> route -- out of scope -----> abstain -> END
      |
      +- plan -> agent <-+
                |      |
                +- tool calls -> tools
                |
                v
            verify -- verified -----> END
                  +- unattested atoms -> repair -> agent
                  +- guard failure -> policy_repair -> agent
                  +- second failure -----> abstain -> END
```

NODE	DOES	MODEL
route	Deterministic triage. In scope at all? Which tier? An out of scope question short-circuits before any tool planning is paid for.	NO
plan	States intent and steps, emitted as a PlanEvent before any tool runs.	YES
agent	The tool calling loop. Chooses tools and arguments.	YES
tools	Executes against the injected ToolSurface, accumulating every envelope into typed state.	NO
verify	Structural guards first, then the grounding check.	NO
repair	One correction pass, told exactly which atoms were unattested.	YES
policy_repair	One correction pass, told exactly which guard failed and which tool would fix it.	YES
abstain	Builds a specific, actionable Abstention.	NO

The graph takes a ToolSurface and a BaseChatModel by constructor injection and never imports a concrete implementation of either. That is why the whole graph runs against a scripted fake with no dataset and no API key, and it is what makes every guardrail testable: each test scripts the model into the exact failure the guard exists to catch.

Design details worth knowing

route is asymmetric	It refuses only when <i>confident</i> the question is outside crew operations. Anything ambiguous passes through, because a wrong refusal here is unrecoverable while a wrong forward is caught downstream.
plan is a product feature	One model call, no tools bound, structured output. Its purpose is the plan stream event: the controller watches the system decide before it acts. The model may <i>raise</i> the tier and never lower it, because a question that says "what should I do" is Tier 3 whatever the model thinks.
tools deduplicates	Every successful call is keyed for the turn (key order is not a difference, and an argument present as None against one left out is not either). A repeat gets its earlier envelope back. It still emits an envelope and a ToolMessage: suppressing the execution is fine, suppressing the reply strands the tool call and the provider rejects the next request.
budgets	8 tool iterations, 30 seconds wall clock, exactly 1 repair pass. Never two, never a silent pass-through.

The five guards

GUARD	REFUSES	BECAUSE
-------	---------	---------

substance_guard	An empty answer, or a bare refusal with no reason	A refusal has to name what was missing and what the system can answer instead
breach_agreement_guard	An answer that <i>leads with a pass</i> when the tools computed a breach	A controller reads the first line and acts on it
verdict_guard	Any legality claim with no rules-engine envelope this turn	A verdict is computed, never inferred
ranking_guard	Any ranked recommendation with no cover search	Ranking needs every candidate enumerated, rule checked and priced, including the rejected ones
tier_guard	A Tier 2 or 3 answer whose tool calls were all retrieval-only	Retrieval establishes what is, not what follows. This is the guard that stops the system degrading into a fluent lookup with a confident tone.

Each failure names the tools that would fix it, so the repair pass is specific rather than scolding. "Call check_legality for C-3310 on P-2291" is actionable; "be more careful" is not.

BREACH_AGREEMENT_GUARD, AND THE ANSWER THAT PRODUCED IT

Q20 asks whether a 90 minute delay pushes the rostered crew over a limit. simulate_delay computed it correctly: breach=True, 12.75h against a 12.0h FDP limit. The model then wrote:

"The scheduled pairing P-2203 passes all seven rules for every rostered crew member" and corrected itself four sentences later.

Neither existing mechanism could see it. The verifier attests values, and every value in that headline was real. verdict_guard checks that a rules tool *ran*, and six check_legality calls did, all returning pass, because the pairing *as scheduled* does pass. The question was about the delay.

So this guard checks a **relation**, and it checks **ordering rather than presence**: an answer that opens with the breach and later notes what passes is correct and gets through. What is refused is reaching the all-clear first.

Prompts are the last line, not the first

agent/prompts.py is versioned and stamped into the trace, and every constraint in it carries a comment naming where it is *actually* enforced. If a constraint appears in the prompt and nowhere else, that is a bug: a prompt is a request, and a request is not a guarantee.

The system prompt does carry one thing that is genuinely only a prompt, and it earned its place by measurement:

```
4. Stop as soon as the tools have established the answer, and write it. Do not
   confirm a figure a tool has already given you. You are working against a wall
   clock: a controller gets nothing at all if the turn runs out, so an
   unnecessary call is not caution, it is the most likely way to lose the answer
   you already had.
```

Q26 got the right answer from scan_duty_headroom on its third call and then made four

more confirming it. Q21 had its verdict by the ptn and made three more after. Both ran out of clock holding a correct answer. Adding that instruction took Tier 2 from 12 of 14 to 13, and average latency from 19.5s to 15.3s.

The provider seam

agent/providers.py is the only module in the repository that names a model vendor. The graph is written against BaseChatModel. Precedence is anthropic, then openai, then ollama, selected by the *presence* of a key, never by a live probe (an HTTP probe would put a round trip on the latency budget of every turn).

Three vendor quirks live there so the graph stays neutral:

1. **The default model id must follow the provider.** claude-sonnet-5 sent to Ollama is a 404 on the first turn.
2. **Sampling must be pinned per vendor.** Anthropic's current models reject sampling parameters outright; Ollama defaults to temperature 0.8, which would make the same question answer differently on consecutive asks.
3. **with_structured_output has no portable method.** Ollama's default json_schema made the tool-capable models ignore the schema and answer in prose, failing the parse. function_calling binds.

THE COST OF PRECEDENCE BY PRESENCE

An *invalid* key takes the system down. An unusable ANTHROPIC_API_KEY is currently in .env.local and every agent turn fails with a 401 in under a second, with zero tool calls. CREWOPS_LLM_PROVIDER=ollama overrides it. **Check crewops health before demoing anything.** If Provider reads anthropic, every agent answer will 401.

Thread memory

Two stores in one SQLite file at .crewops/memory.db. LangGraph's AsyncSqliteSaver holds message state so a follow-up resolves against what came before and a thread survives a restart. A separate turns table holds every settled Reply as JSON, because a controller reviewing a decision wants the answer that was given with its verification report, not the message history that produced it. That is the audit trail.

Conversation names come from the *question*, not the answer's headline, in five words with the identifier first, computed by a lookup table with no model involved. Naming from the headline meant typing "hey" produced a conversation called "This is a crew operations desk assistant".

10 The verifier

Deterministic. No model call, no network, no clock, no randomness. Same input, same report, forever. Four steps.

1. Extract

extract.py scans prose for nine kinds of atom: number, duration, currency, date, time,

extract.py scans prose for nine kinds of atom: number, duration, currency, date, time, identifier, station, rule id, aircraft type.

Every pattern runs over the whole string, then overlaps are resolved by **earliest match, and at equal start the longest**. That ordering is what stops INR 18,500 from also yielding the bare numbers 18 and 500, and what stops the tail VT-DXB from yielding the station code DXB.

List enumerators and markdown furniture are blanked to spaces (preserving offsets) before scanning, so a numbered list does not read as a stream of unattested integers.

THE MOST INSTRUCTIVE BUG IN THE PROJECT

Language models write typographically. gpt-oss renders a crew id as C-3310 with U+2011 NON-BREAKING HYPHEN so it will not wrap. The extractor scanned for an ASCII hyphen, did not find one, and fell through to the bare integer 3310. A number in no tool output cannot be attested, so the grounding check **rejected answers that were correct**, whose figures were all genuinely in the tool results.

It cost four of sixteen Tier 1 questions and it read exactly like the model being stupid. It was us. Folding typographic characters to ASCII before scanning recovered three answers and took grounding from 10/16 to 15/16.

This is not a loosening. Identifiers still compare exactly: C-3310 is still not C-3301. The fold is one character to one character, so every span offset the scanner reports stays valid, and a test pins the other direction: the fold may only recover an identifier that was genuinely written, never manufacture one from prose.

The lesson worth keeping: *an over-strict guard fails in the direction that looks responsible*, which makes it easy to misattribute to the model.

2. Attest

Two channels, counted separately so the gap between them stays visible.

Fact channel (primary)	Every Fact on every successful envelope. Facts are typed, so unit="hours" registers the duration equivalence as well as the bare number, and the derivation string is re-scanned so the operands inside it are attested too.
Payload channel (fallback)	Every scalar reachable in the payload, the trace, the citations and the call arguments of a <i>successful</i> envelope. Still deterministic output, so nothing the model invented can enter this way. What is lost is citation quality, not safety, which is why the report counts payload-only attestations separately: that count is a punch list of Facts the tool layer should be emitting.

Two registration rules deserve their own note:

- **Arguments of a successful call are attested.** `find_crew(base="BLR")` returning `ok=True` makes BLR attested, because the deterministic layer accepted that filter and computed against it.
- **Arguments of a failed call are not.** A lookup for C-9999 returning `ok=False` is exactly

the case where the model made the identifier up.

3. Normalise

The single source of truth for "are these two renderings the same fact". It has no dependency on the rest of the system on purpose, because the eval scorecard's grader imports it directly: there is one normaliser in this repository, so the verifier and the grader cannot disagree.

KIND	CANONICAL FORM	SAME FACT	DIFFERENT FACT
Duration	whole minutes, half up	61.33 / 61.33h / 61h20m / 61 hours 20 minutes	61.3h
Number	2dp decimal string	61.33 / 61.330	61.3
Currency	digits only	18500 / INR 18,500 / ₹18,500	18000
Date	YYYY-MM-DD	2026-09-15 / 15 Sep / September 15	2026-09-16
Identifier	exact, case folded	c-3310 / C-3310	C-3301

That two-minute window on durations is the entire tolerance budget: 61.33h is 3679.8 minutes rounding to 3680, and 61h20m is 3680 exactly. 61.3h is 3678, two minutes away, and therefore a different fact.

4. Decide

Any atom with no attestation becomes an UnattestedAtom carrying its surrounding sentence. The turn gets exactly one repair pass naming the offending atoms. A second failure abstains. Never a third pass, never a silent pass-through.

The allowlist

Four entries, each with a written justification, and a test that fails if it grows. That test is a deliberate speed bump: *a permissive allowlist is how a grounding check quietly stops working*. Nothing fails, the guarantee just evaporates.

- 1. **Three-letter uppercase tokens that are not station codes.** UTC, FDP, ETA, SMS and so on. Note what is deliberately absent: every real station code. "Flights depart HYD" is a claim about the data even when HYD exists.
- 2. **The rule count, in a sentence about rules.** "All seven rules were checked" states a property of the system, not of the dataset. "Seven crew are affected" still needs attesting.
- 3. **A small positional ordinal after a structure word.** "day 1 of P-2291", "option 2". Bounded three ways: a fixed short list of preceding words, values 1 to 9 only, and bare integers only, so "day 1.33" is still checked.
- 4. **A year inside an attested date.** Implemented in attest.py rather than here, which is strictly safer: 2026 is safe only because some tool returned a 2026 date this turn.

What it does not catch

Recorded so nobody assumes more than it delivers.

- **Spelled-out numbers above twelve.** "sixty one point three three hours" is not extracted.
- **Relations between attested atoms.** This is what the graph's guards exist to cover.
- **Unit swaps on an attested value.** If 61.33 is attested, "61.33 minutes" passes. Catching this produces false rejections on correct answers, which is a worse failure.
- **Coincidental collision.** A wrong number that happens to equal an unrelated number in a large payload passes. Running with `require_fact_attestation=True` removes this class entirely, at the cost of requiring complete Fact coverage from the tool layer.

THE BLIND SPOT WORTH NAMING OUT LOUD

Seven names in `crew.json` are shared by two crew each. Ask "is A. Nair legal for P-2291" and an implementation that resolves a name to the first match returns a legality verdict about the wrong person, with correct arithmetic, full rule traces and a grounding check that *passes*, because every figure in it is genuinely attested. It is simply about someone else.

This is the sharpest lesson in the whole exercise. The central mechanism, verifying that every atom is attested, defends against invented facts and is completely blind to correctly computed facts about the wrong entity. Naming that limitation is worth more than pretending the guard is comprehensive.

The defence is a different one: name resolution returns all matches and abstains on more than one, with `AbstentionReason.AMBIGUOUS_REFERENT`. Never resolve a name to one person silently.

11 The offline resolver

Everything runs with no API key. `resolve/` matches the question against a fixed set of shapes, calls the same tools, renders through deterministic templates, and passes through **the same structural guards and the same grounding check**. Replies are marked `mode=DETERMINISTIC` and the UI shows the badge, because hiding which path answered would overstate the system.

Four stages

1. **Canonicalise.** A controller types `C1042, bangalore, dx 412`. The offline path matches fixed shapes that expect `C-1042, BLR, DX412`. Only the *question* is rewritten, never the answer, so nothing here can put a value in front of a controller that a tool did not produce. `Reply.question` keeps what they actually typed.
2. **Triage.** Shared with the agent's route node, so the two modes never disagree about what is answerable.
3. **Match an intent.** 17 shapes, each naming the tools it will run and the entities it needs. An intent that matches but is missing a required entity produces a *specific* abstention ("name the crew member") rather than a generic one.
4. **Render, guard, verify.** Templates assembled from Fact values and prose deterministic code authored. No repair pass, because a template producing an unattested figure is a bug in the template, not something to negotiate with.

A greeting is answered. Trivia is refused.

These look similar and are not. Refusing trivia is the system working: an advisor that answers general knowledge will also confidently answer a duty-hour question it should decline. Refusing a greeting is the system looking broken, and it is the first thing anyone types.

Matching is therefore strict: **every** token must be a pleasantry and the whole thing at most four tokens. That is what keeps "hey, who is on reserve at BLR" a real question, which is how a controller under pressure actually types.

Why this is a first-class product, not a fallback

THE CLEANEST PROOF OF THE WHOLE THESIS

The deterministic path has **no model in it at all** and still answers Tier 1 at 100% accuracy in single-digit milliseconds, with the same facts, the same arithmetic and the same grounding line reading "4 of 4 figures traced to a tool result". If the model were producing the facts, this path could not exist.

It became the measuring instrument too. Every agent result is scored against the offline result on the same question, which is how the Q20 verdict inversion was caught.

THE UNCOMFORTABLE VERSION

If the two modes score the same, the honest conclusion is *not* that the fallback is impressive. It is that the agent is not earning its place, and this is the mirror image of the "AI decorating a lookup" failure the 20% criterion names. `make eval --mode` both prints that comparison and prints a warning panel when the two modes disagree on nothing at all.

12 The console

Next.js 16, React 19, Tailwind 4. Four routes: the Advisor (a conversation), the morning brief, Operations, and Architecture. **No answering logic anywhere.**

The SSE contract

Twelve event types over `POST /api/chat`. EventSource cannot issue a POST and the chat request carries a body, so the client reads the response stream by hand: `fetch` plus a `ReadableStream` reader, a decoder in streaming mode, and a buffer that survives chunk boundaries landing mid-frame.

EVENT	CARRIES	THE UI SHOULD
run_started	thread_id, question, mode	open the turn, show the mode badge
plan	intent, tier, steps	show what the system intends to do

tool_call	tool, args, label	append a live step chip
tool_result	ok, latency_ms, summary, envelope	settle the chip, stock the evidence drawer
trace	step	append to the reasoning trail
token	text	stream provisional prose, <i>visibly provisional</i>
verifying	atom_count	show the grounding check running
verification	report	show verified, repaired or rejected
abstain	abstention	render the refusal card, not an error
reply	the full Reply	replace provisional prose with the settled answer
done	total_ms	close the turn

The ordering guarantee: `verification` always arrives before `reply`, and `reply` always before `done`. Tokens are provisional until `reply` lands, and the UI must not present them as final. That is the difference between a system that looks honest and one that is. The runner owns the monotonic seq counter and emits `verification` itself rather than letting the verify node do it, so it fires exactly once and always immediately before the reply, including on paths where the graph abstained and never verified anything.

Reconnection policy

A connection that fails *before the first event arrives* is retried with backoff: nothing has been observed, so a retry is free. A connection that fails *mid stream* is **not** retried, because re-running the turn would run the agent a second time and a controller would have no way to tell a resumed answer from a fresh one. The caller is handed a recoverable error and offers a retry the controller chooses.

Interface decisions with a reason behind them

The answer is not in a bubble	The question is, because a question is something somebody said. The answer is set as the page's own body text at full measure. Two facing bubbles make a transcript between equals; what is actually happening is that a controller asks and the system produces a <i>document</i> . A document does not go in a speech bubble, and once the frame is gone the answer can use full width for a table or an option card.
An abstention is not an error state	No red, no warning triangle, no apology. Calm ground, the reason named, what was missing, what was established anyway, and the questions that would work instead. "Would a controller find this trustworthy" is answered by what the system does when it <i>cannot</i> answer.
Fact chips	Hovering a figure in the prose highlights the Fact that attests it and shows its derivation. The linking does string matching only over surface forms generated from Fact values the API supplied. If a number appears and no Fact carries it, the

code leaves it alone. It does not guess.

Result sets are tables	Projected in Python by agent/tabulate.py, not formatted by a model. Every cell is copied out of a payload field a tool already attested, so a Table is exactly as grounded as its payload. Asking a model to format a table would put a hundred figures through a language model on the way to the screen.
The offline path replays its tools	The resolver ran six real tools and measured six real latencies. Those are replayed as events after it returns, each carrying the latency that tool actually took. Nothing is invented and no progress is faked: a run that finishes in four milliseconds emits its rows in four milliseconds.

Bugs the code comments record

These are worth reading because each one was invisible in the obvious check:

- **Four typefaces "did not change anything"** because the next/font variables were on <body> and read on :root, its parent. Custom properties inherit downward and never up, so both tokens resolved to empty and every surface fell back to the UA stack. The woff2 files were requested and served the whole time. The check that finds it is getComputedStyle(document.body).fontFamily, not the network tab.
- **Rename and delete failed in the browser and worked under curl**, because the CORS allowlist read GET, POST, OPTIONS and those routes are PATCH and DELETE. The request never reached the route, the store or the UI. A test now asserts the allowlist against the router itself.
- **Timestamps read hours early.** _utcnow() stripped the tzinfo, so every timestamp serialised without an offset, and ECMAScript parses a date-time with no offset as local time. The conversation list said "5h ago" about a conversation created one second before. On a product whose dataset is emphatically all-UTC, that is the same class of defect as a report time in the wrong zone.
- **A switched conversation received another one's answer.** Start a chat, type, then click an existing thread while the answer is still streaming. Fixed with a generation counter: every switch increments it, every async callback captures the generation it started under, and a stale writer is dropped.

13 Evaluation

make eval runs all 38 questions through the advisor and grades every answer against the shipped key. Four outcomes, and the distinction between two of them is the whole point.

OUTCOME	MEANING
CORRECT	Every primary fact in the key is asserted, and no verdict is inverted
PARTIAL	Some primary facts asserted, none contradicted
ABSTAINED	The system declined, with a reason. Never counted as a failure.

WRONG

Facts missing beyond the threshold, or a verdict inverted

An abstention is never a failure, because the problem statement says answering ten questions correctly and declining the eleventh scores higher than answering all eleven with three wrong. A scorecard that folded abstentions into the wrong column would push the project in exactly the wrong direction.

A **verdict inversion** is graded worse than a miss and flagged unsafe on its own line. Saying a candidate is legal when the key says they breach is an operational error, not a shortfall in recall, and no amount of correct surrounding detail redeems it.

Grading a natural language answer

String equality is useless. The key says "RULE-DUTY-02: would exceed 60h/7d by 1h20m on 2026-09-15 (total 61.33h)" and a good answer says "C-2087 would be 1.33 hours over the 60 hour limit on 15 September". Those are the same answer.

So the grader imports `crewops.verify.normalise` directly and adds exactly two things the verifier must not have: a flattener that renders answer-key JSON as scannable text, and **grading tolerance**. The verifier is strict on purpose; a grader needs the opposite bias, because marking a correct answer wrong pushes the team to optimise the wrong thing.

Three expansions, each safe here and unsafe in the verifier: duration to number, currency to number, and flight id to flight number (the keys mix DX412-2026-09-15 and DX412 for the same leg).

Measured state

Deterministic mode, all 38 questions

TIER	N	CORRECT	PARTIAL	ABSTAINED	WRONG	ACCURACY	GROUNDED	AVG / P95
Tier 1	16	15	0	1	0	100%	15/16	4ms / 5ms
Tier 2	14	6	1	6	1	75%	8/14	8ms / 14ms
Tier 3	8	2	0	6	0	100%	2/8	32ms / 91ms
All	38	23	1	13	1	92%	25/38	9ms / 23ms

Coverage is the weak column, not correctness. 13 of 38 questions are declined, and that is a deliberate trade rather than an accident.

The model bake-off, Tier 1, same 16 questions, same keys

MODEL	CORRECT	ABSTAINED	WRONG	GROUNDED	AVG	P95
deepseek-v4-flash	16	0	0	16/16	6.5s	12.1s
deterministic (no model)	15	1	0	15/16	4ms	5ms
gpt-oss:120b	13	1	1	15/16	7.2s	10.0s
glm-5.1	13	3	0	13/16	16.9s	27.4s

qwen2.5:7b

unusable: returns an empty tool_calls list for a bound schema

HOW TO READ A SCORECARD HONESTLY

kimi-k2.6 reports **100% accuracy**. It abstained on 11 of 16 and got the remaining 5 right: correct whenever it answers, and useless at a desk. Its p95 of 63.2 seconds is past the problem statement's "a 45 second response is not a decision aid" line outright.

Always quote accuracy next to coverage. Never quote either alone.

These models do not honour temperature 0 strictly. deepseek scored 16, 15, 16 on three identical passes. **No single agent-mode number is stable enough to quote on its own.** Quote a range, or run three times and say so.

Tier 2 through the agent, as the fixes landed

STAGE	CORRECT	ABSTAINED	WRONG	AVG / P95
deterministic (no model)	6	6	1	7ms / 11ms
agent, first measurement	5	8	1	23.5s / 32.5s
after the correctness fixes	10	4	0	18.0s / 30.6s
after cutting round trips	12	2	0	19.5s / 32.9s
after the stopping rule	13	1	0	15.3s / 27.2s

Correctness more than doubled and the verdict inversion is gone. **None of the three fixes was prompting.**

1. **A verdict inversion, closed.** `simulate_delay` computed the Q20 breach correctly and emitted it as a Fact but not as a `RuleTrace`. The assembled Reply therefore carried 31 PASS traces and zero BREACH, for a question whose answer is a breach. The tool now emits the FDP evaluation as a trace, *pass or breach*, because a rule that was checked and cleared is evidence and silence leaves a controller unable to tell "checked, fine" from "never checked".
2. **breach_agreement_guard.** The new relation check described in section 9.
3. **Round trips.** Found by printing the *arguments* of every call rather than the tool names. Q27 called `list_reserves` four times, two of them byte-identical apart from key order. The graph had always executed every call in a message together and nothing had ever asked the model to put more than one there. And the planner's tool list was hand-written and six tools out of date, missing `scan_duty_headroom` and `earliest_report`: a planner that does not know a tool exists plans around it, which is how "which crew have 45 or more duty hours" became `find_crew` plus 150 calls to `get_duty_clocks`.

Held-out generalisation

`internal/held_out_scenarios.json` is judging material, gitignored, and the tests skip cleanly when it is absent. Only **safety** is asserted: no verdict inversions and no crashes. The score is *printed and never asserted*, because a threshold on held-out judging material is overfitting with extra steps

14 What fails, and why

make test reports **587 passed, 7 failed, 15 xfailed, 2 deselected** in 3.4 seconds. The two deselected are live-provider tests, excluded by default so the suite stays offline and free.

The repository notes say the seven failures are "all traced to intent matching in `resolve/`". That is directionally right but imprecise. They are **two distinct bugs**, both in `resolve/`, and the ops engine underneath is provably correct in both cases.

Bug A: the closure renderer never names the flights

Affects Q19 (77% recall), Q29 (0%) and scenario S3.

`simulate_station_closure` returns a **dict** payload with the `ImpactReport` nested under "impact". But `render.py` does:

```
def _payload(envelopes, wanted):
    for envelope in envelopes:
        if isinstance(envelope.payload, wanted):    # dict is not ImpactReport
            return envelope.payload
    return None
```

So `_render_impact` silently falls through to `_render_generic`, which prints the trace line and the count facts and nothing else. The flight ids are sitting in `payload["affected_flights"]` and never reach the text:

```
Q29 expected  ["DX461-2026-09-19", "DX462-2026-09-19"]
Q29 produced  "2 flights touch HYD inside the window. Delaying each to reopen
              plus a 30 minute turnaround keeps 2 of them within crew FDP..."
              0% recall
```

The engine is completely correct: `test_disruption.py` asserts all 13 S3 rows byte for byte and passes. This is a rendering drop, not a computation error.

Bug B: scenario narratives match the wrong intent

Affects S1, S2, S6 and the flagship test.

S2's narrative reads *"Captain C-1042 calls in sick at 05:00Z on 15 Sep for his 2-day pairing P-2291..."*. That matches the `sick_impact` intent at priority 75, not `cover_options` at priority 90, because it contains "calls in sick" and no "what should I" or "ranked options".

So the system answers the Tier 2 impact question correctly, while the S2 answer key is a Tier 3 recommendation key with 6 ranked options and 19 exclusions. Hence **14% recall on an answer that is not wrong, just to a different question**.

```
S2 produced:  "Captain C-1042 is unavailable from 2026-09-15 (sick call). That breaks
              1 pairing (P-2291) and leaves 6 legs without a Captain. 486 passengers
              are exposed on the day of the absence and 972 across the whole broken
              pairing. Uncrewed: DX412, DX413, DX588, DX589, DX590, DX591."
```

S2 key wants: 6 ranked options with costs, 19 exclusions with reason strings, and the expected choice C-3310 at INR 18,500.

The ops engine reproduces S2 perfectly:
test_cover.py::test_s2_options_and_exclusions_reproduce_exactly asserts all 6 options and all 19 exclusion strings and passes. Only the resolver's *plan* is wrong.

A latent third issue in the same file

The cover_options intent builds find_cover_options with exclude_crew_ids but **never passes for crew_id**. A question naming a person and a date with no pairing therefore hits the tool's own error path. That is the shared cause behind the six Tier 3 abstentions.

Bug C: Q12 duplication

```
matching = [f.flight_no for f in flights if f.block_hours == extreme]  # no dedupe

-> "The longest block time is 2.75h, on DX401, DX589, DX402, DX401, DX402,
    DX588, DX401, DX589, DX402, DX401, DX402, DX588, DX401, DX589, DX402,
    DX401, DX402, DX588, DX401, DX589, DX402."

21 entries. Four distinct. Expected: DX401, DX402, DX588, DX589.
```

It grades **CORRECT** only because matching is containment-based, so the grader is hiding a real presentation bug. Fix is dict.fromkeys(...).

WHAT THIS MEANS FOR TOMORROW

All three are in resolve/, all three are localised, and the deterministic core underneath is verified against the shipped keys in every case. Bug A is small and buys two questions plus a scenario. Bug B is the flagship. If there is an hour available, that hour is worth more than anything else in the repository.

The honest failure analysis

docs/FAILURE-ANALYSIS.md grades every failure as **safe** (the system declines and says why) or **unsafe** (the system produces a fluent, confident, wrong operational instruction). The second is worse than having no system at all.

#	FAILURE	SAFE OR UNSAFE
F1	Simultaneous disruptions: the same reserve assigned to two aircraft	UNSAFE critical
F3	Ambiguous crew referent (seven shared names)	UNSAFE IF UNGUARDED
F7	The 09-14 duty clock overlap, if somebody "fixes" it	UNSAFE IF FIXED
F2	Compound questions answered in half	SAFE
F4	A question needing a rule outside the seven	SAFE
F5	Data the pack does not contain	SAFE

F5	Data the path does not contain	SAFE
F8	Aggregation over the schedule	SAFE
F9	The offline path is narrower than the agent path	SAFE
F10	RULE-FLT-03 implemented and never exercised	LATENT
F12	The grader itself being wrong	META

F12 is worth mentioning unprompted, because a failure analysis that examines the system and not the instrument measuring it is incomplete. The scorecard's own tests caught two bugs in it before the system it grades even ran: `reduce_expected` treated any `rank` field as a ranking position, and Q01's reserve list carries `"rank": "Captain"` as a job title, so twelve required reserves collapsed to whichever name sorted first alphabetically.

15 Drift and loose ends

Small things found while reading, none of them load bearing, all of them the kind of detail a thorough judge might notice.

<code>.env.local</code>	Contains a live-looking GitHub personal access token, plus Tavily, Gemini and Ollama keys. It <i>is</i> gitignored so nothing has leaked, but it sits at the repository root. Rotate these before screen-sharing anything.
<code>web/src/lib/contracts.ts</code>	The hand-written TypeScript mirror lists 17 tools; Python has 24 . Missing: <code>find_pairings</code> , <code>find_crew_at_risk</code> , <code>aggregate</code> , <code>get_cost_rates</code> , <code>simulate_delay</code> , <code>scan_duty_headroom</code> , <code>earliest_report</code> , <code>plan_joint_cover</code> . Consequence is cosmetic (tool chips lose their tier colour) but the contracts package says in writing that the mirror moves in the same change.
<code>crewops health</code>	Never shows snapshot, hub or currency. Both <code>cli.py</code> and <code>server/deps.py</code> guard on <code>isinstance(summary.payload, dict)</code> , but <code>get_world_summary</code> returns a <code>WorldSummary</code> model. The branch is dead, which is why the documented health output only shows four rows.
<code>DATA-MODEL.md counts</code>	States 210 roster crew references in two places. The shipped data has 206 (25 x 6 plus 14 x 4). <code>test_store.py</code> already caught it and documents the discrepancy in a comment.
<code>allowlist size</code>	<code>allowlist.py</code> 's docstring and <code>verify/CLAUDE.md</code> both say "exactly three entries" while <code>ALLOWLIST_SIZE = 4</code> and four are documented. <code>AGENT-DESIGN.md</code> correctly says four.
<code>_role_to_cover</code>	Defaults to "Captain" when it cannot determine the role from the arguments. Defensible for this dataset (every scenario is a captain except S5) but it is a silent assumption on a path where guessing is expensive.

04/09/2026, 20:12	Crew Ops Advisor Field Guide
<code>web/README.md</code>	Still the default create-next-app boilerplate. Several code comments reference "web/README.md, Contract gaps", which does not exist.
<code>architecture-view.tsx</code>	Labels the grounding guard's package as <code>narrate/</code> , <code>verify/</code> . There is no <code>narrate/</code> package; it is <code>verify/</code> alone.
<code>make lint</code>	<code>ruff</code> check passes. <code>ruff format --check</code> wants 41 files reformatted and has for some time. Worth one isolated commit, not worth doing tonight.
Thread memory	The checkpoints table has 0 rows and has never had any, because every turn logged so far ran in deterministic mode, which bypasses the graph entirely. Multi-turn follow-up is implemented and undemonstrated. Say this rather than let it be discovered on stage.

16 Judge questions

How do I know the model isn't doing the arithmetic?	<code>uv run pytest tests/test_boundary.py -v</code> . It walks the import graph of seven packages and fails if a model client is reachable at any depth, twice over: an AST walk and a live import check. It also asserts the mirror, that <code>agent/</code> does import one, because a boundary with nothing on the other side of it is decorative.
How do I know it isn't making the numbers up?	Every answer prints a grounding line: <code>4/4 figures traced to a tool result</code> . The verifier extracts every number, duration, currency amount, date, time, identifier, station code and rule id from the drafted prose and attests each against what the tools returned. Unattested atoms get one repair pass, then the turn abstains.
Show me it failing safely.	Ask it something out of scope, or run Tier 1 on <code>gpt-oss</code> . Every scorecard prints "No verdict inversions. Every failure failed safely." Then show the abstention card: the typed reason, what was missing, what was established anyway, and three questions that would work instead.
Does it work without the internet?	<code>uv run crewops ask "..."</code> <code>--offline</code> . Zero model calls, three milliseconds, and the grounding line still reads 4 of 4. That is the whole deterministic path, and it answers Tier 1 at 15 of 16.
What does it do badly?	Tier 3 answers 2 of 8. The flagship scenario currently grades wrong at 14% recall, and the cause is intent matching in the offline resolver, not the rules engine. Thread memory is written and barely exercised. Simultaneous disruptions would be answered unsafely without the joint planner, so that is the one place the system abstains deliberately rather than composing two independent searches.

Isn't this just a rules engine with a chat box?

That is the sharpest version of the question and the honest answer has a number attached. `make eval --mode both` prints the questions where the two modes disagree. Tier 2 is currently 13 of 14 through the agent against 6 of 14 offline: the agent is composing tools in orders the fixed shapes do not have a rule for. If that table were empty the criticism would land, and the harness prints a warning panel saying so.

Why seven rules and not more?

Because the problem statement says the seven provided rules are the full regulatory scope. `RuleId` is a closed `Literal`, so an eighth is a type error. The answer keys *do* use an eighth constraint (double booking) and it is implemented as a `FeasibilityIssue`: blocking, but honestly labelled operational rather than regulatory, never given a `RULE-` id, and never listed in `rules_checked`.

How would this handle crew PII in production?

The unusual thing this architecture already does: **the model never sees the dataset**. It sees tool results, so the set of fields crossing the boundary to a third-party inference provider is enumerable, and it is enumerated in `tools/payloads.py`. In production: pseudonymise at the tool boundary (the agent reasons about C-1042 perfectly well without knowing who that is, and the UI rehydrates the name locally); keep medical certificate *detail* out of the model's half entirely, since the rules engine needs to know whether a certificate is valid on a date and not why one is not; log the evidence ledger rather than the prompt, because the Fact list is the audit record a regulator wants and it is already structured; and apply purpose limitation to reachability data, which is location adjacent.

Would this scale?

The boundary gets *stronger* with scale: the more data there is, the worse an idea it is to put it in a prompt. The tool surface is already a query interface backed by a projection, so the same tools run against a real database by changing the store. What needs work: candidate enumeration is a linear scan over eligible crew, correct and instant at 150 and needing an index and an early cutoff at 15,000, because a controller needs the top five options and not a complete ordering. What would change: the seven-day and 28-day windows are recomputed per candidate per day, which is correct and cheap here and becomes an incrementally maintained running total at scale, which is what a real rostering system does.

What is the business case?

Deliberately *not* an annual savings figure, because the problem statement says base rates are synthetic and uncalibrated, and a system whose whole thesis is that it never invents a figure cannot open its business case with one. The defensible claims: the per-decision cost delta, which is computed and citable (\$4 re-crews one leg for 75,000 against 250,000 to cancel it, a 3.3x difference on one decision); the downstream consequences surfaced that a controller cross-referencing screens would have to find by hand; and the fact that the reasoning trail makes a decision reviewable at shift change.

BEFORE ANYTHING ELSE

cd api && uv run crewops health. If Provider reads anthropic, every agent answer will 401 in under a second. Prefix agent commands with CREWOPS_LLM_PROVIDER=ollama until that key is fixed or deleted. This is the single most likely way the demo breaks.

1. Open with no API key at all

```
cd api
uv run crewops ask "Who is on reserve at BLR on 2026-09-15?" --offline

tier 1 | deterministic, no model
12 reserve(s) on call for 2026-09-15.
Grounding: verified 4/4 figures traced to a tool result
1 tool calls, 0 model calls, 3 ms
```

This is the strongest opening because it answers "is this just a wrapper on a model" before anyone asks it. Zero model calls, three milliseconds, and the grounding line still says 4 of 4.

2. The same question through the agent

```
CREWOPS_LLM_PROVIDER=ollama uv run crewops ask "Who is on reserve at BLR on 2026-09-15?"
```

Same answer, now with a visible plan and tool trace. Slower, because a model is in the loop. The point is that the *answer* did not change, only the language and the route to it.

3. The thirty seconds that wins it: C-3305

Not C-1042. Every team gets handed C-1042 by the problem statement and every team will demo it, and it demonstrates competence. C-3305 demonstrates that you understood *why the problem is hard*.

```
Can reserve C-3305 cover the full pairing P-2291?
```

Show day 1 passing at 59.50h against the 60h limit. Then show day 2 at 68.25h, over by 8h15m. Then say the sentence:

Day one's cover duty is itself inside day two's seven-day window. An implementation that checks the first day and stops returns "yes" with correct arithmetic for the day it checked, and nothing about that answer looks like a partial check.

Second candidate for the same slot: **C-5837**, whose rest conflict is two days after the cover and is invisible to any same-day check.

4. The boundary, proved rather than drawn

```
uv run pytest tests/test_boundary.py -v
```

Show it next to the /architecture page. A diagram *asserts* the boundary; make boundary *proves* it. Both together is the strongest version.

5. The proactive brief

```
uv run crewops brief 2026-09-15
```

14 items for that date, 2 critical. The first is C-5417, rostered illegally on 2026-09-19 because a recurrent training certificate expires on 2026-09-17. Deterministic, no model call on this path. Not asked for by the rubric, listed under optional enhancements.

6. Close on the failure analysis

Most teams will present capability. Presenting a known limitation, with the arithmetic showing exactly why it fails and what you would do about it, is worth more and costs nothing you have not already written. The problem statement says it in terms: *"Honest failure analysis scores well; overstating capability scores badly."*

Lead with F1, the simultaneous disruption. It is the only failure in the whole question set that would produce a confident wrong operational instruction rather than a refusal, and the system abstains on it deliberately.

Commands, for the terminal

```
make install      # uv sync for the API, pnpm install for the web
make dev          # API on :8000 and web on :3000
make test         # 587 pass, 7 known failures, 3.4 seconds
make golden       # parity against the shipped answer keys
make eval         # scorecard across all 38 questions
make check        # ruff, mypy, boundary test, tests
make boundary     # just the boundary test

uv run crewops ask "." --offline
uv run crewops brief 2026-09-15
uv run crewops health

CREWOPS_LLM_PROVIDER=ollama uv run python -m crewops.eval.scorecard --mode both
```

Compiled from a full read of the repository at commit cc49cde, with the test suite and the deterministic path exercised directly. Every figure quoted here came from the shipped dataset, a passing test, or a command run against this checkout.

Sources in the repository: docs/DATA-MODEL.md for the decoded dataset, docs/AGENT-DESIGN.md for the graph, docs/CONTRACTS.md for the seam, docs/FAILURE-ANALYSIS.md for the honest version, DECISIONS.md for why each choice was made and what it cost.

